

AUTOMATED REVIEW RATING SYSTEM

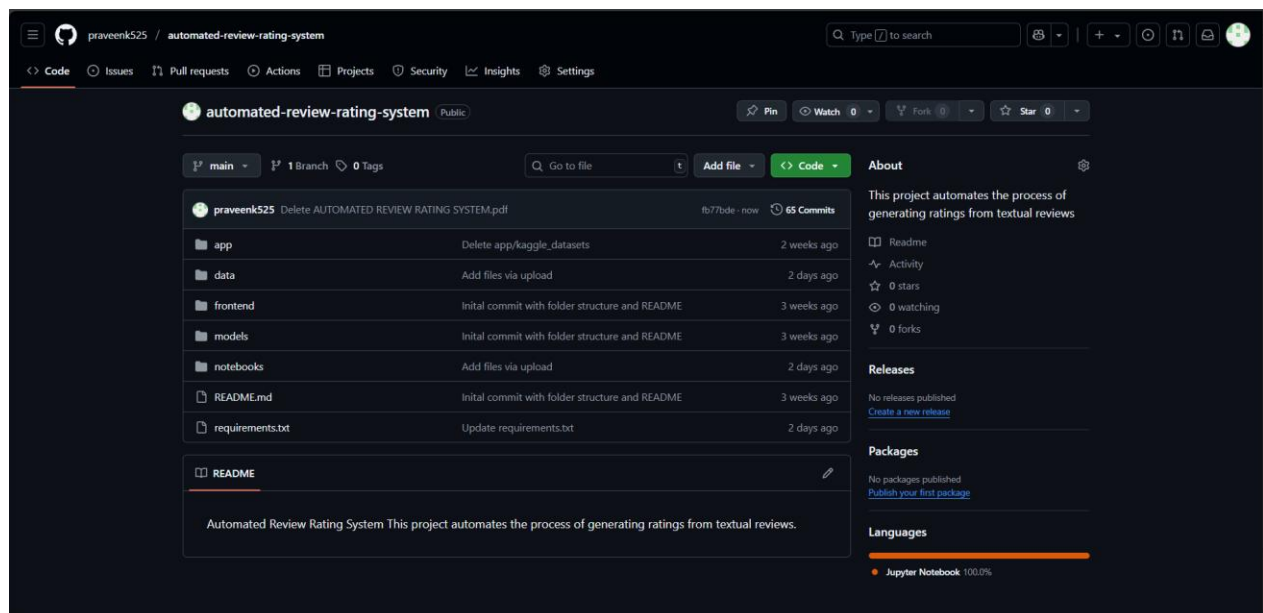
PROJECT OVERVIEW

This project automates the process of generating ratings from textual reviews. an AI-based system that automatically predicts the numerical rating (e.g., 1 to 5 stars) of a product, service, or business based solely on user-generated textual reviews. This aims to reduce rating manipulation, standardize feedback, and assist platforms in quickly gauging sentiment at scale.

ENVIRONMENT SETUP

- Installed python with necessary libraries such as numpy, pandas, matplotlib, scikit-learn, seaborn, nltk
- Colab

GITHUB PROJECT REPOSITORY



DATA COLLECTION

Data was collected from Kaggle and google datasets.

The project uses 10 diverse datasets across industries to ensure model generalization:

- Women's Clothing Reviews
- TripAdvisor Hotel Reviews
- Laptop Product Reviews
- IMDB Movie Reviews
- Amazon Product Reviews (including various categories)
- Plus 5 other domain-specific review datasets (e.g., food, electronics, services, etc.)

Each dataset contains:

- Review text
- Corresponding user rating (1–5 stars)
- Optional fields: title, product ID, date, reviewer info

S.No.	Dataset Name	Domain	Source	Approx. Size	Columns	Links
1	Recipe reviews	Food & Hospitality	Kaggle	~18182	15	https://github.com/praveenk525/automated-review-rating-system/tree/main/data/kaggle_datasets
2	Reviews& ratings	E-commerce	Kaggle	~1535	8	https://github.com/praveenk525/automated-review-rating-system/tree/main/data/kaggle_datasets
3	Walmart	E-commerce	Kaggle	~300	6	https://github.com/praveenk525/automated-review-rating-system/tree/main/data/kaggle_datasets
4	Womens clothing	E-commerce	Kaggle	~23486	11	https://github.com/praveenk525/automated-review-rating-system/blob/main/data/kaggle_datasets/Womens%20Clothing%20E-Commerce%20Reviews.csv
5	Amazon reviews	E-commerce	Kaggle	~4915	12	https://github.com/praveenk525/automated-review-rating-system/tree/main/data/kaggle_datasets

6	imdb_tvshoows	Movies & Tv	Google	~8319	5	https://github.com/praveenk525/automated-review-rating-system/tree/main/data/kaggle_datasets
7	Laptops	Electronics	Kaggle	~24113	7	https://github.com/praveenk525/automated-review-rating-system/tree/main/data/kaggle_datasets
8	review_details	E-commerce	Kaggle	~2501	32	https://github.com/praveenk525/automated-review-rating-system/tree/main/data/kaggle_datasets

EXTRA DATASETS:

- Train(E-commerce) from Kaggle with size of ~ 60000, columns =3
Link: https://github.com/praveenk525/automated-review-rating-system/blob/main/data/kaggle_datasets/train.csv
- Tripadvisor (Food & Hospitality) from Kaggle with size of ~ 20491, columns=2
Link: https://github.com/praveenk525/automated-review-rating-system/blob/main/data/kaggle_datasets/tripadvisor_hotel_reviews.csv

All datasets are merged into one

Merged_dataset link=https://github.com/praveenk525/automated-review-ratingsystem/blob/main/data/merged_datasets/Merged_dataset.csv

DATA PREPROCESSING

Preprocessing is the first and most important step in any machine learning or data science project. It involves cleaning, transforming, and organizing raw data into a format that your machine learning model can understand and learn from effectively.

Load and Inspect Datasets

- Load all 10 datasets (e.g., Women's Clothing, TripAdvisor, etc.)
- Inspect structure: check column names, data types, sample records
- Verify presence of review and rating columns `pd.read_csv()` → Load CSV files into

a DataFrame `pd.read_excel()` → For Excel files

```
# View first few rows

df1.head()

# View last few rows

df1.tail()

# Check column names

df1.columns

# Get DataFrame shape (rows, columns) df1.shape

# Check column data types

df1.dtypes # Full summary

info df1.info()

# Summary statistics (for numeric columns) df1.describe()
```

Remove Unnecessary Columns

- Drop columns not required for prediction, such as:
 - user_id, product_id, timestamp, title, image_url, etc.
- Keep only relevant columns: usually review_text and rating

Remove Null or Missing Values

- Remove rows with null values in review or rating

```
#Check for missing values in each column

print(df.isnull().sum())

# removing null values

df.dropna(inplace=True)

# Remove rows where review_text or rating is missing df.dropna(subset=['review_text',
'rating'], inplace=True)
```

Remove Duplicates and Conflicting Reviews

- **Exact duplicates:** same review and rating keep one
- **Conflicting reviews:** same review text, different ratings remove all

```
#check duplicates print(df.duplicated().sum()) # Remove all duplicate rows  
df.drop_duplicates(inplace=True)
```

Clean Text Data

- Convert text to lowercase
- Remove:
 - URLs
 - Punctuation
 - Numbers and special characters
 - Extra spaces

Stopwords Removal

Stopwords are common words in a language (like *"the"*, *"is"*, *"at"*, *"which"*, *"and"*) that do not add significant meaning to text and are usually removed during text preprocessing in NLP tasks.

Why Remove Stopwords?

- They don't help in understanding sentiment or meaning.
- They inflate feature space and slow down training.
- Removing them improves model accuracy and efficiency.

What is Lemmatization?

Lemmatization is the process of reducing a word to its root form (lemma) based on its dictionary meaning and part of speech.

Example:

- "running" → "run"
- "better" → "good"
- "was" → "be"

Lemmatization uses a vocabulary (dictionary) and morphological analysis to correctly identify the base form of a word.

What is Stemming?

Stemming is a simpler, rule-based process that chops off prefixes or suffixes to get to the root word. It doesn't care about grammar or word meaning.

Example:

- "running" → "run"
- "flying" → "fli"
- "studies" → "studi"

Why Lemmatization is Better (in most ML tasks):

- Returns actual words (important for readability and clarity).
- More grammatically correct results.
- Handles irregular words better (e.g., "better" → "good").
- Reduces noise compared to stemming.

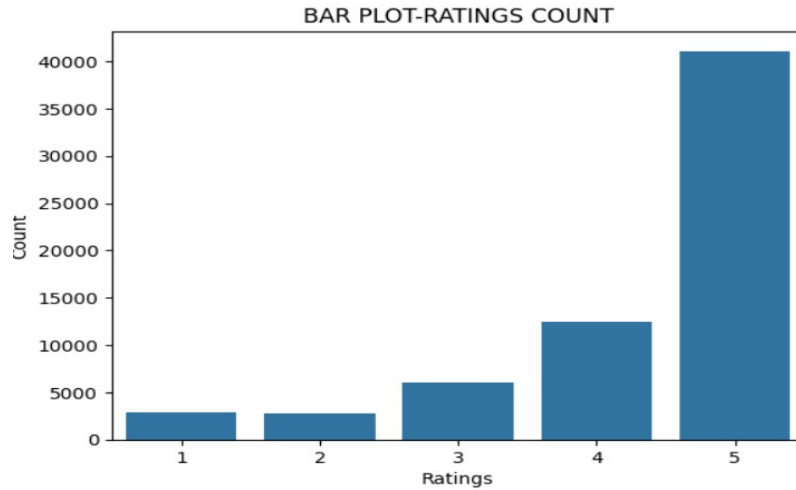
DATA VISUALIZATION

Data Visualization is the process of representing data and information visually using charts, graphs, and other visual elements (like maps or word clouds) to help people understand patterns, trends, and insights in data.

BAR CHART

A bar chart represents categorical data with rectangular bars. The length of each bar is proportional to the value it represents.

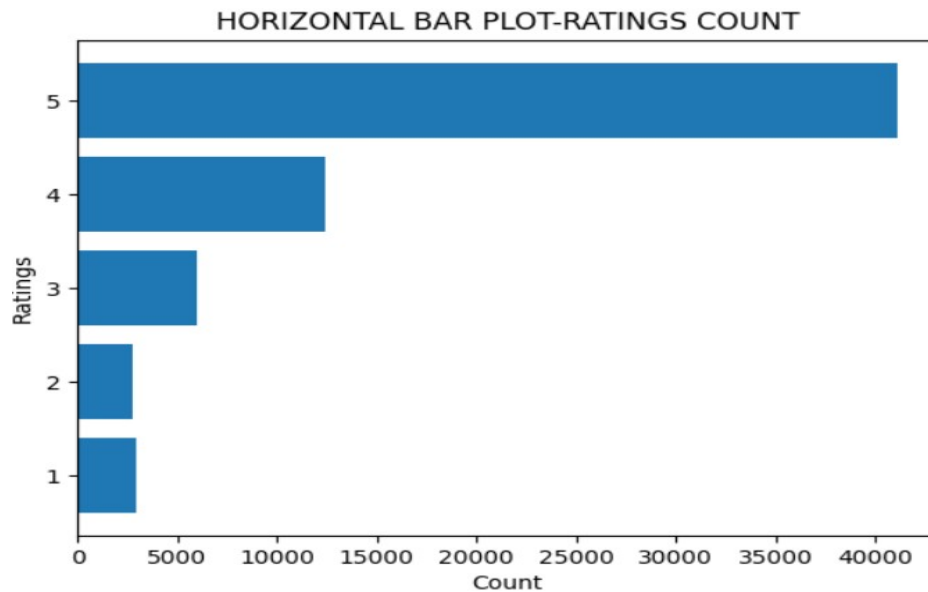
- Use: To show the number of reviews in each star rating (e.g., how many 1-star, 2-star, etc.).
- Number of reviews per rating (1 to 5).



Horizontal Bar Chart (HBar)

A horizontal bar chart is like a bar chart, but the bars extend horizontally instead of vertically.

- To show average values like review length for each rating in a clean, readable format.



Pie Chart

A pie chart is a circular chart divided into slices to illustrate proportions of categories.

- To show the percentage of each rating (how much of the data is 5-star, 4-star, etc.).



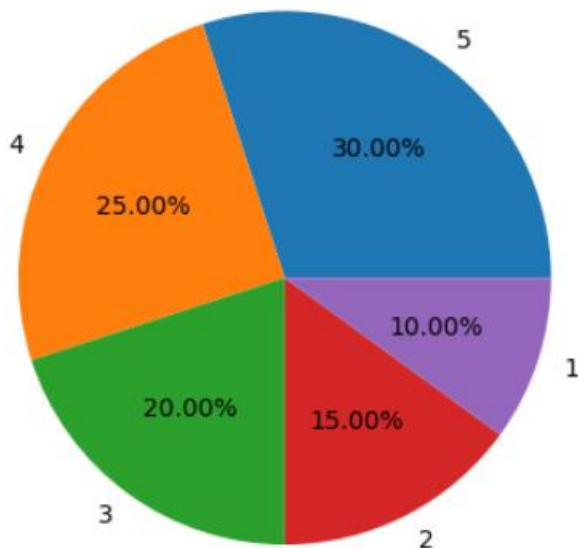
IMBALANCED DATASET

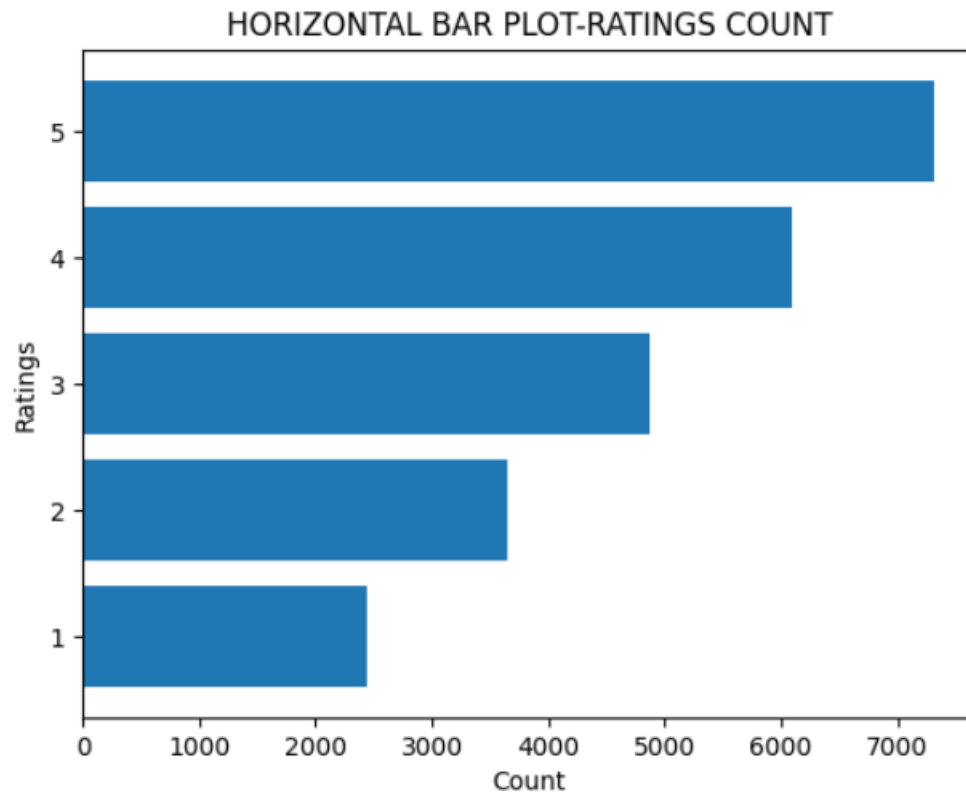
An imbalanced dataset means the classes in your target variable are not represented equally — one or more classes have many more samples than others.

This is common in classification problems, especially in review rating datasets, fraud detection, medical diagnosis, etc.

Imbalanced Link: https://github.com/praveenk525/automated-review-rating-system/blob/main/data/balance_imbalance_datasets/imbalanced-data.zip

Imbalanced dataset with 1 star-10%, 2 star-15%, 3 star-20%, 4 star- 25%, 5 star- 30%





Train-Test Split

What is Train-Test Split?

In machine learning, we split the dataset into:

- **Training Set** → Used to train the model (e.g., 80% of data)
- **Testing Set** → Used to evaluate the model's performance (e.g., 20% of data)

This helps ensure the model can generalize to unseen data.

Why Use stratify?

If the dataset is imbalanced (e.g., more 5-star reviews than 1-star), a random split could cause:

- Training set to have fewer examples of rare classes.
- Testing set to not represent all classes equally.

Stratification ensures:

- The class distribution (percentage of each rating) is the same in both train and test sets.

TF-IDF Vectorization

What is TF-IDF?

TF-IDF (Term Frequency – Inverse Document Frequency) is a feature extraction technique used in NLP to convert text into numerical vectors.

It assigns higher weight to important words in a document and lower weight to common words.

Why Use TF-IDF?

- Removes bias toward frequent but unimportant words (e.g., "the", "and").
- Captures the importance of words in relation to all reviews.
- Works well for text classification tasks like review rating prediction.

How It Works

1. TF (Term Frequency)

Measures how often a word appears in a document.

$$TF = \frac{\text{number of times the term appears in the document}}{\text{total number of terms in the document}}$$

2. IDF (Inverse Document Frequency)

Measures how important a word is across all documents.

$$IDF = \log\left(\frac{\text{number of the documents in the corpus}}{\text{number of documents in the corpus contain the term}}\right)$$

3. TF-IDF SCORE

$$TF-IDF = TF * IDF$$

Why We Chose the Algorithm

We selected Logistic Regression for the initial implementation because:

- It is a simple yet powerful linear classification algorithm that works well for text classification tasks.
- It is computationally efficient, making it suitable for large datasets.
- Logistic Regression provides probabilistic outputs, which can be used for confidence scores in predictions.
- It has a proven track record in sentiment analysis and review rating prediction tasks.

- It works well with TF-IDF vectorized textual data due to its ability to handle high-dimensional sparse features.

Preprocessing Steps Used

Before training the model, the following preprocessing steps were applied:

1. **Loading and Inspecting Data** – All 10 datasets (Women's Clothing, TripAdvisor, Laptops, IMDB, Amazon Products, etc.) were loaded and merged.
2. **Removing Unnecessary Columns** – Dropped columns unrelated to review text or rating (e.g., product IDs, usernames).
3. **Removing Null Values** – Eliminated rows with missing reviews or ratings.
4. **Removing Duplicates** – Removed duplicate reviews to avoid bias.
5. **Text Cleaning** – Lowercasing, removing special characters, numbers, and extra spaces.
6. **Stop Words Removal** – Removed common words (e.g., "the", "is") that do not contribute to meaning.
7. **Lemmatization** – Reduced words to their base form for better consistency.
8. **TF-IDF Vectorization** – Converted text into numerical features using Term Frequency–Inverse Document Frequency to capture word importance.
9. **Train-Test Split with Stratification** – Ensured class distribution consistency between training and testing dataset.

Model Training Logic

- The cleaned and vectorized dataset was split into 80% training and 20% testing sets.
- Logistic Regression was trained using Scikit-learn with the following steps:
 1. Initialize the model with `logistic regression(max_iter=5000,c=1.0,solver='lbfgs')`
 2. Fit the model to the training data (`model.fit(X_train, y_train)`).
 3. Generate predictions for the test data (`model.predict(X_test)`).
- Hyperparameter tuning was performed using GridSearchCV to find the optimal regularization parameter (C) and solver type.

NCE

Evaluation Results and Interpretation

The model's performance was evaluated using:

- **Accuracy Score** – Percentage of correct predictions.
- **Confusion Matrix** – To visualize correct vs. incorrect predictions per rating class.
- **Classification Report** – Showing precision, recall, and F1-score for each class.

Accuracy: 0.5665859564164649

Classification Report:				
	precision	recall	f1-score	support
1	0.59	0.77	0.67	460
2	0.49	0.53	0.51	644
3	0.45	0.41	0.43	900
4	0.50	0.47	0.48	1130
5	0.72	0.69	0.71	1409
accuracy			0.57	4543
macro avg	0.55	0.58	0.56	4543
weighted avg	0.56	0.57	0.56	4543

